

ELDERLY MEDICATION REMINDER

Laura Marín Ocón and María del Mar Madrid Delgado

figma: [click here](#)

github: [click here](#)

trello: [click here](#)

1. Introduction	3
1.1. Problem context	3
1.2. Purpose of the system	3
1.2.1. Current System	3
1.2.2. Proposed System	3
1.3. Product vision	3
2. Design Overview	4
2.1. User Personas	4
2.2 Scenarios	6
2.3 User stories	7
2.5 User Features	9
2.6 Sprint Evidence	10
3. System Architecture	16
3.1 Hardware Architecture	16
3.2. Software Architecture	16
3.2.1.Key interfaces	20
4. Data Design	20
4.1 Conceptual Data Model	20
5. Security Management	22
6. Testing	23
7. Prototyping	29
8. Conclusion	33

1. Introduction

1.1. Problem context

Older people who need daily medications struggle to keep their schedules straight. It's easy to forget a pill or get the dosage wrong. Misreading a label can also lead to serious health risks. Most patients rely on memory or handwritten notes and family help, but that makes mistakes more likely. Plus, doctors and families rarely stay in sync, so nobody knows for sure if the treatment plan is being followed.

1.2. Purpose of the system

This system's goal is to create a digital platform to assist elderly persons with their medication management. Doctors may write and manage their own prescriptions, caregivers will have a way to monitor adherence by patients, and patients will receive timely reminders. By providing all medication-related data on a single platform, this system will enhance communication between key stakeholders (patients, caregivers, providers) increasing accuracy with fewer missed doses, and improving overall communication.

1.2.1. Current System

Nowadays, our method of managing medications usually consists of manual and separate methods, including written medication schedules, verbal medication instructions, and reminder applications. These methods do not integrate healthcare providers and patients together, resulting in a lack of tracking medication adherence and updates to medication prescriptions. Because of fragmented methods, there is a high risk of miscommunication between patients and providers, missing doses or obtaining correct medication information and no accountability for the correct use of medication.

1.2.2. Proposed System

An upcoming system is a multi-platform healthcare app, a mobile app for patients and web-based dashboard for doctors and caregivers that are connected via a single backend platform. Doctors are able to create and edit prescriptions; caregivers will be able to track patient activity, while patients will receive automatic reminders to take their medicine. The system will provide stakeholders with real-time updates within the system, improve coordination between them, and make managing medication schedules more reliable.

1.3. Product vision

For elderly patients who need to take medication regularly and often forget doses,
whose treatment can be compromised by missed medication and lack of monitoring,
the *ElderCare Medication Management System*
is a digital healthcare platform
that provides medication reminders, prescription management, and adherence monitoring through a simple and accessible interface.

Unlike basic reminder apps, **our product** allows doctors to manage prescriptions, caregivers to monitor adherence, and stores a secure patient medication history in one system.

2. Design Overview

2.1. User Personas

Persona 1 – Elderly Patient (Primary User)

Name: Margaret O'Connor

Age: 76

Occupation: Retired teacher

Location: Lives alone in a small town apartment

Background

Margaret takes many daily medications for blood pressure, arthritis, and diabetes. She sometimes forgets whether she has taken her medication, which causes anxiety. Her daughter lives in another city and checks on her regularly.

Margaret has a smartphone but prefers very simple interfaces with large text and clear instructions.

Goals

- Remember to take medication on time
- Avoid taking the wrong medication
- Maintain independence without relying too heavily on family

Frustrations

- Forgetting whether medication has been taken
- Confusing medication schedules
- Small or complicated mobile interfaces

Needs

- Clear reminders with large, readable text
- Simple confirmation button for medication intake
- A system that informs caregivers if medication is missed

Technology Comfort

Low-Medium. Uses smartphone for calls and basic apps.

How the System Helps

The medication reminder app sends clear notifications at the correct times and allows Margaret to confirm when medication has been taken. If she forgets, the system alerts her caregiver.

Persona 2 – Caregiver

Name: Laura O'Connor

Age: 45

Occupation: Accountant

Relationship: Daughter of Margaret

Background

Laura works full-time and cannot always visit her mother. She wants to be sure that her mother is taking medication correctly, but does not want to constantly call and check.

Goals

- Ensure her mother takes medication correctly
- Receive alerts if medication is missed
- Monitor medication adherence remotely

Frustrations

- Worrying about missed medication
- Lack of visibility into daily routines
- Having to manually check on medication schedules

Needs

- A dashboard to view medication adherence
- Notifications when doses are missed
- Access to current prescriptions and medication history

Technology Comfort

High. Uses mobile apps and web services regularly.

How the System Helps

The caregiver dashboard allows Laura to monitor medication adherence and receive alerts when doses are missed, enabling her to respond quickly if needed.

Persona 3 – Doctor

Name: Dr. James Patel

Age: 45

Role: Doctor

Background:

Dr. Patel is in charge of several elderly patients who are required to have daily medications, he wants to ensure that they are following their prescriptions.

Goals

- Create and manage prescriptions
- Track patients treatment
- Keep organized medical history organized

Frustrations

- Patients forgetting medication
- Lack of follow-up visibility between visits

- Paper-based prescription tracking

Needs

- Easy prescription management
- Access to patient adherence data
- Secure patient information system

Technology Comfort

High

How the System Helps

The system helps doctors manage prescriptions, monitor patient adherence to medication schedules, and maintain an organized and secure record of patient treatment.

2.2 Scenarios

Scenario 1 – Doctor Creates a Prescription

The Patient database can be accessed by Dr. Patel, He uses his secure user account to log in to the system. Dr. Patel is now able to search for and view the patient profile of Margaret O'Connor.

Dr. Patel will then be able to write Margaret a prescription for her blood pressure medication. In addition to writing the prescription, Dr. Patel will also need to enter medication name, dosage, as well as dosage frequency (1 Pill at 8:00 AM Every Morning).

The system will save the prescription and automatically create a medication schedule for Margaret. Because of this medication schedule, Margaret will also receive reminders according to this prescription.

Scenario 2 – Patient Receives Reminder and Confirms Medication

A notification is sent from an app that sends reminders when to take medication to Margaret's phone at 8:00 AM. Once the app opens, it shows a basic layout of the medication name and dosage in large fonts. After taking her medication, she presses the confirm medication taken button. Confirmation gets logged into the medication log and updated into the adherence record for her caregiver's reference.

Scenario 3 – Caregiver Monitors Medication Adherence

Laura accesses her caregiver account online to check out the medication adherence dashboard of her mother (Margaret), who is in the hospital. The dashboard shows that Margaret is following the medication schedule as expected (has taken her medicines on-time) over the last seven days.

Laura is very relieved and can continue to be comfortable knowing she doesn't have to call her mother (or go to the hospital) each day for updates on her mother's progress through her monthly treatment program.

Scenario 4 – Missed Dose Alert (Something Goes Wrong)

When Margaret doesn't take her medicine and fails to confirm the application reminder, the system will be able to identify that there is no confirmation after some specified period of time. The system will then generate an automated alert to Laura that Margaret may have missed a dose of her medication.

Laura will be able to view Margaret's medications via the caregiver dashboard, and Laura will call Margaret to remind her to take her medication.

This alert system is an excellent tool for avoiding missed doses, thus enhancing patient safety.

2.3 User stories

1. [SP: 5] As a user, I want to create an account so that I can access the system securely.
2. [SP:3] As a user, I want to log in securely so that I can access my dashboard.
3. [SP: 5] As the system, I want to restrict access by role so that users only see permitted data
4. [SP: 2] As the system, I want encrypted password storage.
5. [SP:5] As a doctor, I want to create a prescription so that medication schedules are generated.
6. [SP: 3] As an elderly user, I want to view today's medication schedule.
7. ed.[SP: 5] As a doctor, I want to update prescriptions so that treatment changes are reflected.
8. [SP: 5] As a doctor, I want to update prescriptions so that treatment changes are reflected
9. [SP: 3] As a doctor, I want to add clinical notes so that patient history is documented.
10. [SP: 5] As a doctor, I want to view adherence statistics.
11. [SP: 5] As an elderly user, I want to receive medication reminders.
12. [SP: 5] As an elderly user, I want to confirm medication intake.
13. [SP: 5] As the system, I want to prevent duplicate intake confirmations.
14. [SP: 5] As a caregiver, I want to view adherence history.
15. [SP: 5] As a caregiver, I want to receive alerts when doses are missed.
16. [SP: 3] As a doctor, I want to record allergies.
17. [SP: 5] As the system, I want to prevent prescriptions that conflict with allergies.
18. [SP: 5] As the system, I want to log all prescription changes.
19. [SP: 3] As the system, I want to store intake logs for traceability.
20. [SP: 5] As the system, I want to maintain structured patient medical history.
21. [SP: 3] As an admin, I want to manage user accounts.
22. [SP: 3] As an elderly user, I want high contrast interface.
23. [SP: 3] As an elderly user, I want large buttons and text.
24. [SP: 5] As the system, I want to handle network failure during confirmation.
25. [SP: 5] As the system, I want to escalate alerts if multiple doses are missed.

2.4 Use Cases

The next set of use cases provide information about the primary interactions between users of the ElderCare Medication Reminder System. Use cases are derived from initially created personas, scenarios and use story and are reflected in expected behaviours associated with every major workflow in this system.

Example 1 – Create Prescription

Field	Content
Use Case	Create Prescription
Primary Actor	Doctor
Description	Doctor creates a medication prescription for a patient
Preconditions	Doctor is logged in; patient exists
Main Flow	1. Doctor opens patient profile 2. Enters medication details 3. Submits prescription 4. System validates and saves prescription 5. Medication schedule is generated
Alternative Flow	Prescription conflicts with allergy → system warns doctor
Postconditions	Prescription stored and active

Example 2 – Confirm Medication Intake

Field	Content
Use Case	Confirm Medication Intake
Primary Actor	Patient
Description	Patient confirms that medication has been taken
Preconditions	Reminder has been triggered
Main Flow	1. Patient receives reminder 2. Opens reminder screen 3. Presses “Taken” 4. System records intake
Alternative Flow	Duplicate confirmation attempted → system rejects
Postconditions	Intake record stored; adherence updated

Example 3 – Monitor Adherence

Field	Content
Use Case	Monitor Adherence
Primary Actor	Caregiver
Description	Caregiver reviews patient medication adherence

Preconditions	Caregiver logged in and assigned to patient
Main Flow	1. Caregiver opens dashboard 2. Selects patient 3. Reviews adherence report
Postconditions	Adherence data displayed

Example 4 – Receive Missed Dose Alert

Field	Content
Use Case	Receive Missed Dose Alert
Primary Actor	Caregiver
Description	Caregiver receives notification of missed medication
Preconditions	Patient misses scheduled intake
Main Flow	1. System detects missed dose 2. Generates alert 3. Sends notification to caregiver
Postconditions	Alert stored and delivered

2.5 User Features

The ElderCare Medication Reminder System includes a set of core Minimum Viable Product (MVP) features required for the first release, along with potential future enhancements for later development.

Feature	Description	Priority
User Authentication and Role Management	Allows users to securely log in and access features based on their assigned role.	MVP
Patient Profile Management	Enables doctors and caregivers to manage patient profiles and medical information.	MVP
Medication Prescription Management	Allows doctors to create and manage digital prescriptions.	MVP
Medication Reminder System	Sends scheduled reminders to patients when medication is due.	MVP

Medication Intake Confirmation	Allows patients to confirm medication intake through the mobile app.	MVP
Caregiver Monitoring Dashboard	Enables caregivers to monitor medication adherence remotely.	MVP
Missed Dose Alerts	Automatically alerts caregivers when medication doses are missed	MVP
Allergy Recording and Safety Check	Validates prescriptions against known patient allergies	MVP
Adherence Tracking and Reporting	Calculates and displays adherence statistics to doctors/caregivers.	MVP
Audit Logging / Medical History Tracking	Records key actions for traceability and historical review.	MVP
Google Sign-In Integration	Provides alternative social login authentication.	Later Feature
Smartwatch / Wearable Notifications	Sends reminders to wearable devices.	Later Feature

2.6 Sprint Evidence

The project utilized an Agile project management method and was tracked utilizing a Trello Board. The Trello Board is set up in a number of ways on columns that represent each stage of development (Backlog, Sprint Planning, In Progress, Review/Testing, and Done).

User Stories fall under three different Sprint types: Sprint 1, Sprint 2 and Sprint 3 and are also grouped by various roles (Doctor, Elderly User, Caregiver, and System). Each User Story has Story Points (SP) associated with it to help estimate the level of effort associated with completing that User Story.

This overall structure allowed our team to monitor our progress, prioritize what features need to be delivered first, and develop the system in increments over time.

Figure 1: Product Backlog in Trello

This figure shows the product backlog, which contains all the user stories identified for the system. The backlog includes functionalities related to different roles such as doctor, elderly user, caregiver, and system.

User stories are written using the standard format “As a... I want...” and include story points (SP) to estimate development effort. This backlog serves as the basis for sprint planning.

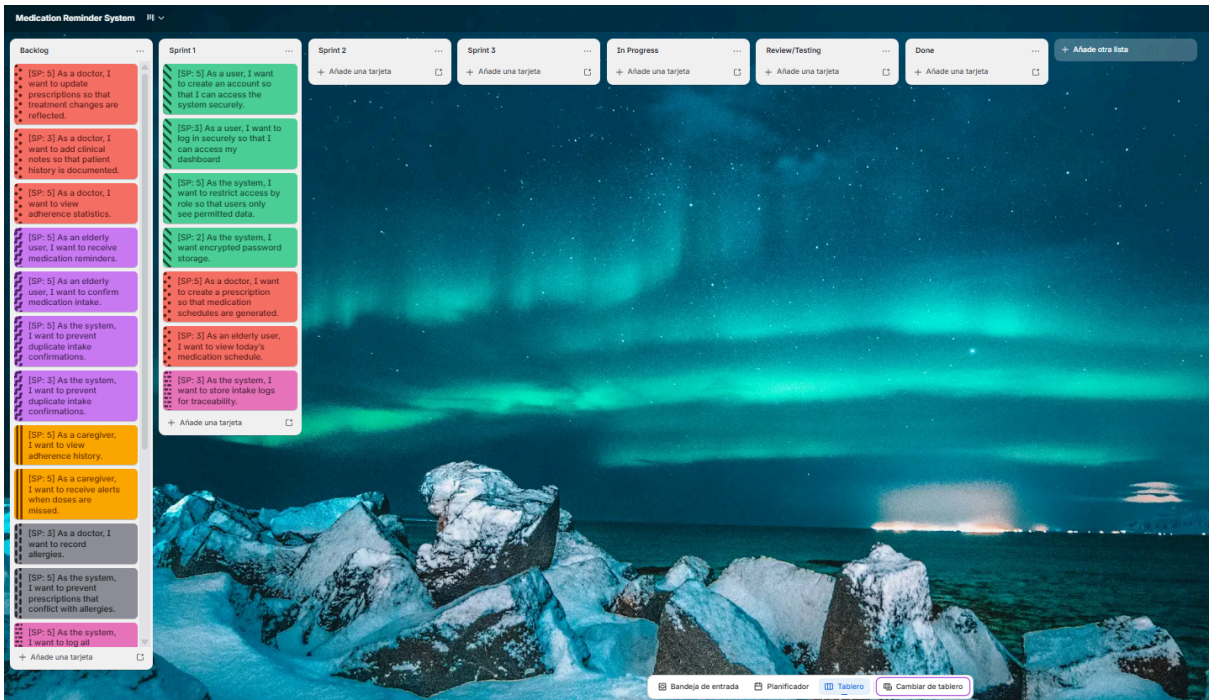


Figure 2: Sprint Planning

This figure illustrates how user stories are distributed across different sprints. Sprint 1 focuses on core functionalities such as authentication and security, Sprint 2 includes features related to prescription management and patient interaction, and Sprint 3 contains advanced and future enhancements.

This organization reflects an incremental development approach.

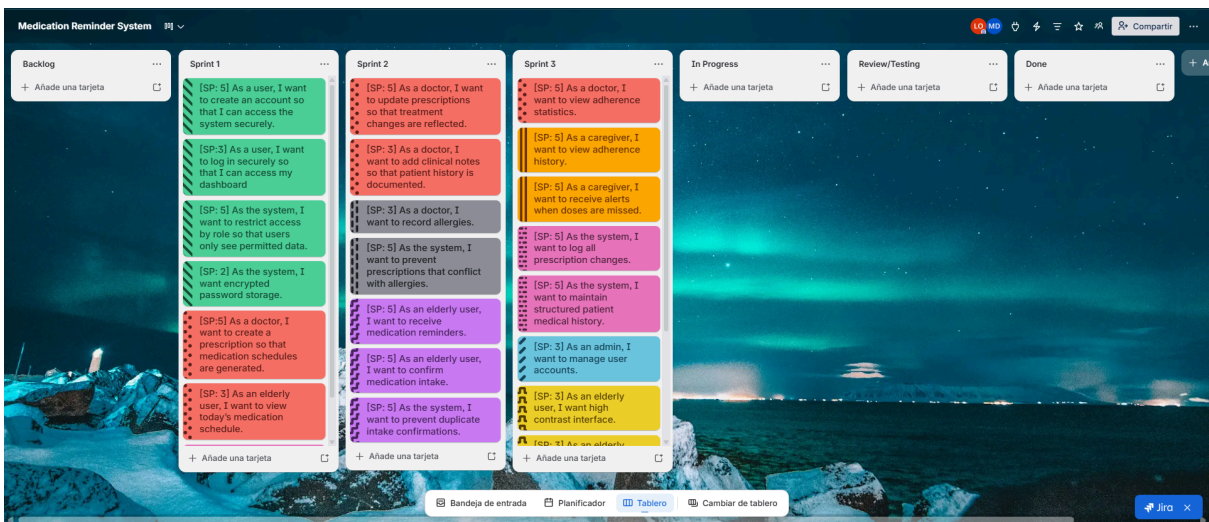


Figure 3: Tasks in Progress

This figure shows the tasks currently under development. These user stories are actively being implemented and represent the ongoing work of the system.

Limiting the number of tasks in progress helps maintain focus and ensures efficient development.

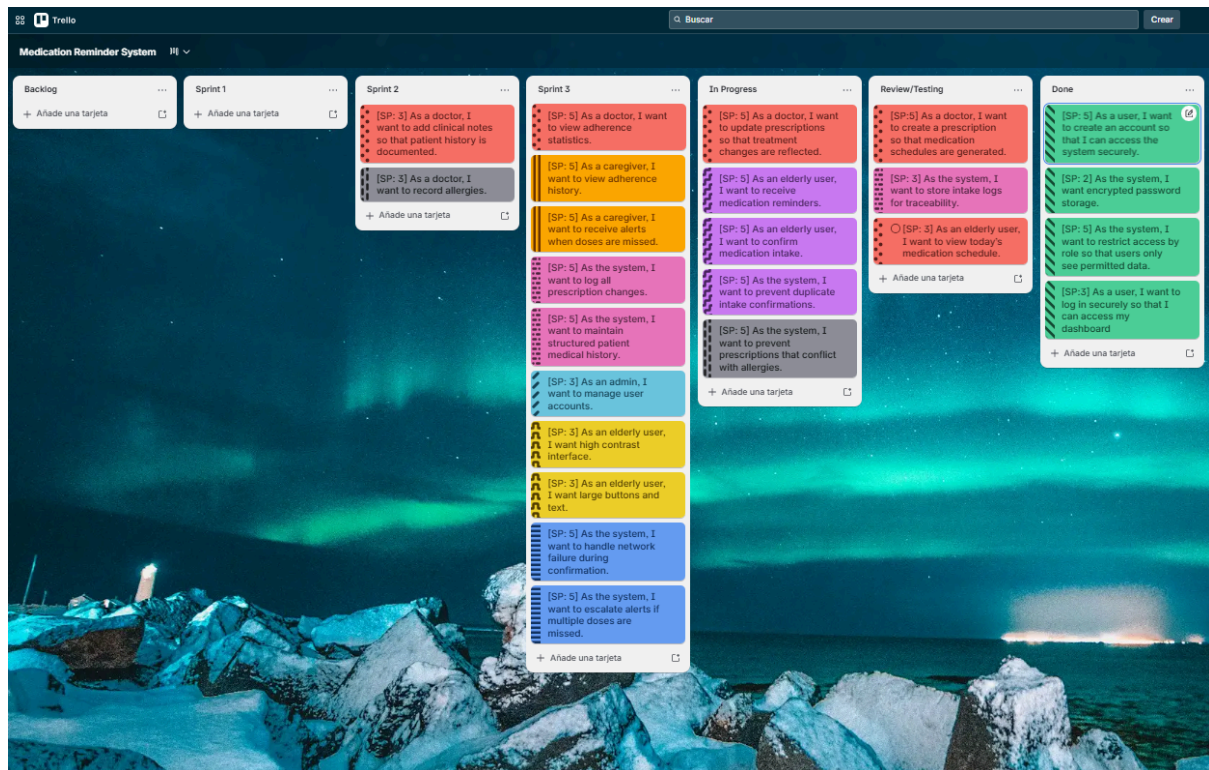


Figure 4: Review and Testing Phase

This figure represents tasks that have been implemented and are currently under testing. These tasks are being validated to ensure they meet the system requirements and function correctly.

This stage includes verification of business logic, system behavior, and integration between components.

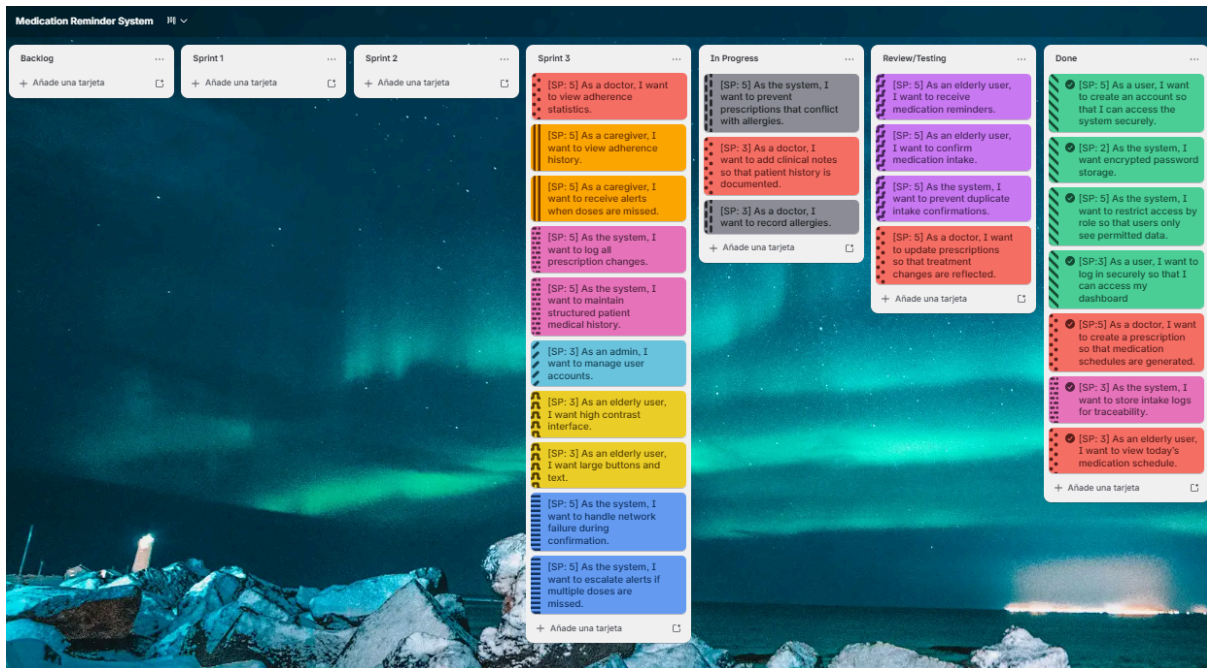


Figure 5: Completed Tasks

This figure shows the user stories that have been fully implemented and validated. These tasks have successfully passed the testing phase and meet all acceptance criteria.

The completed tasks include core system functionalities such as user authentication, role-based access control, and key patient features.

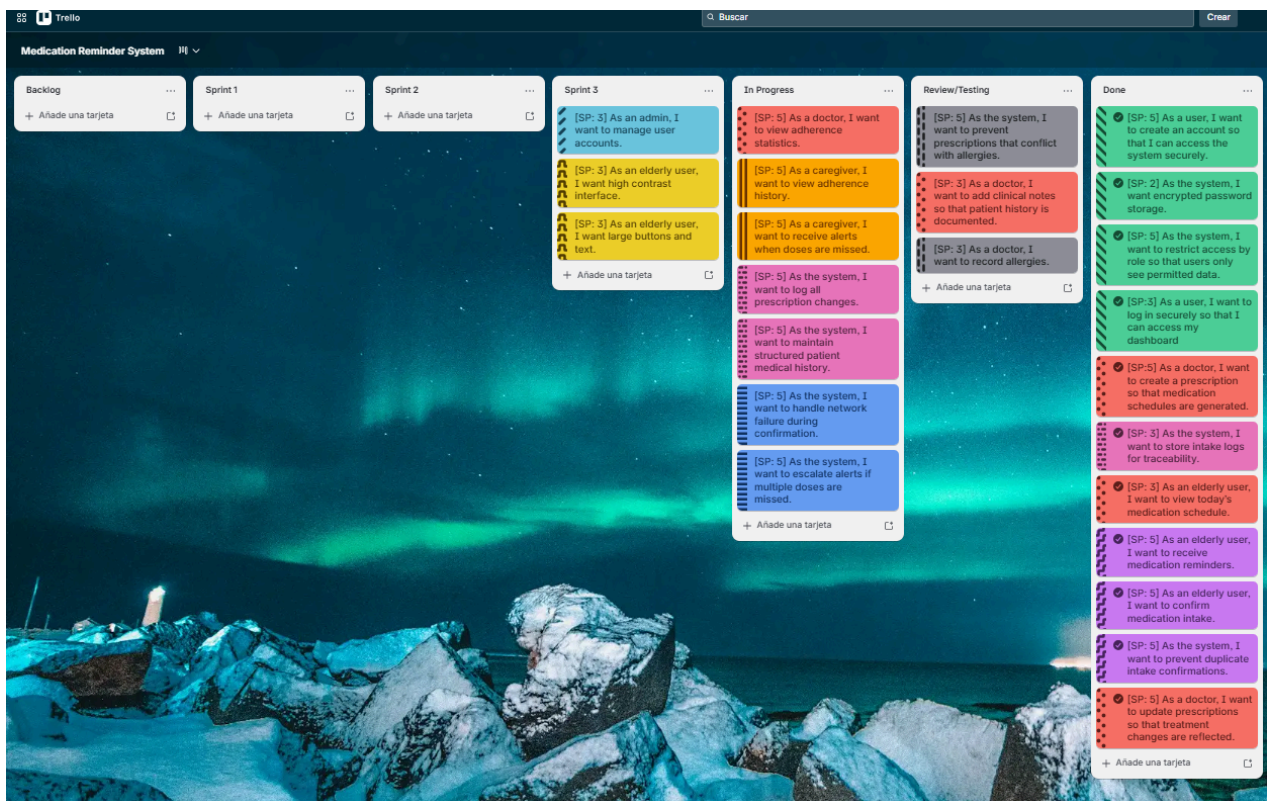


Figure 6: Agile Workflow Overview

This figure provides an overview of the entire workflow in the Trello board. Tasks move from sprint planning to execution through the columns “In Progress”, “Review/Testing”, and “Done”. This workflow reflects an Agile development process, allowing continuous tracking of progress and iterative delivery of system features.

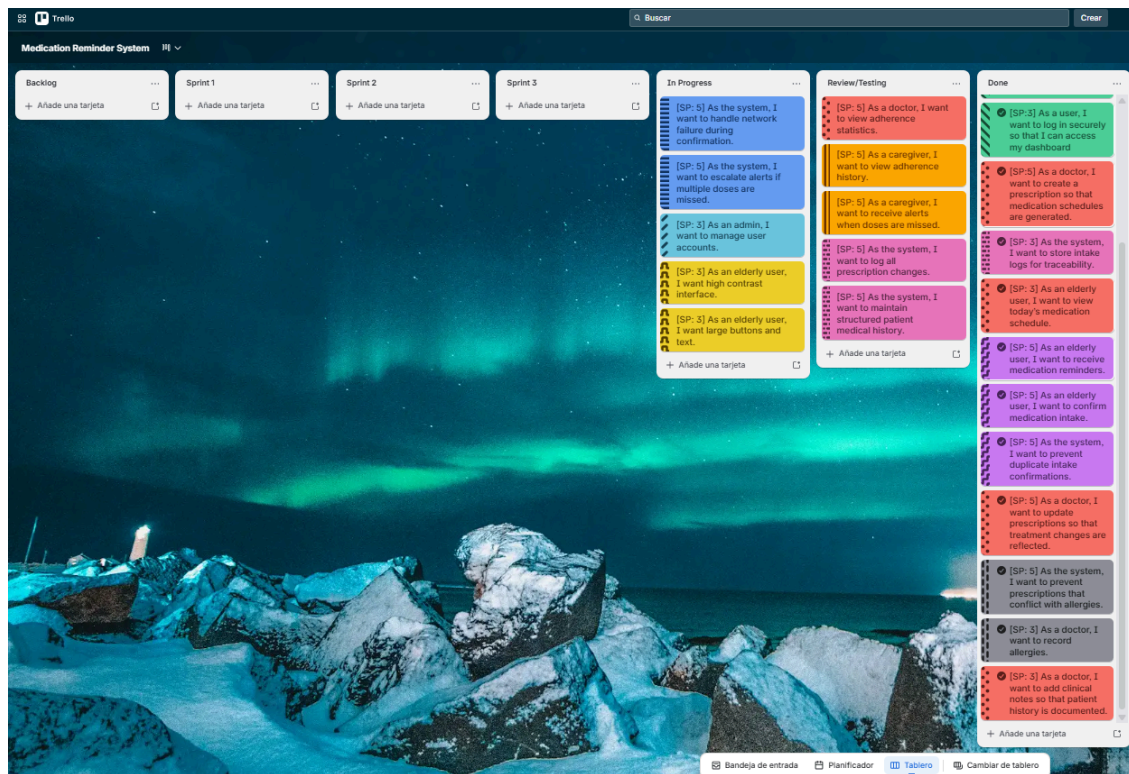


Figure 7: Detailed Task Distribution

This figure provides a more detailed view of the task distribution across the workflow columns. It highlights how different types of user stories (functional, system-level, and user interface tasks) are managed simultaneously. The board demonstrates how multiple features are developed in parallel while maintaining clear organization and traceability.

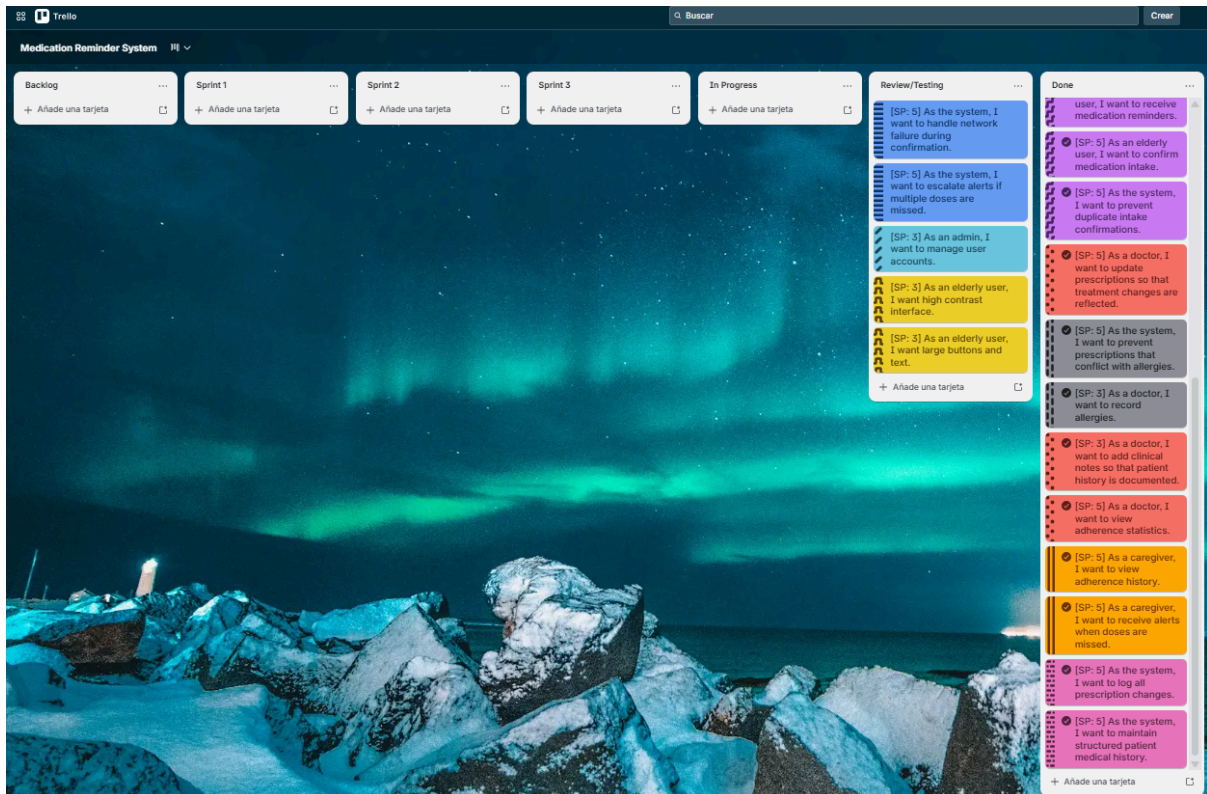
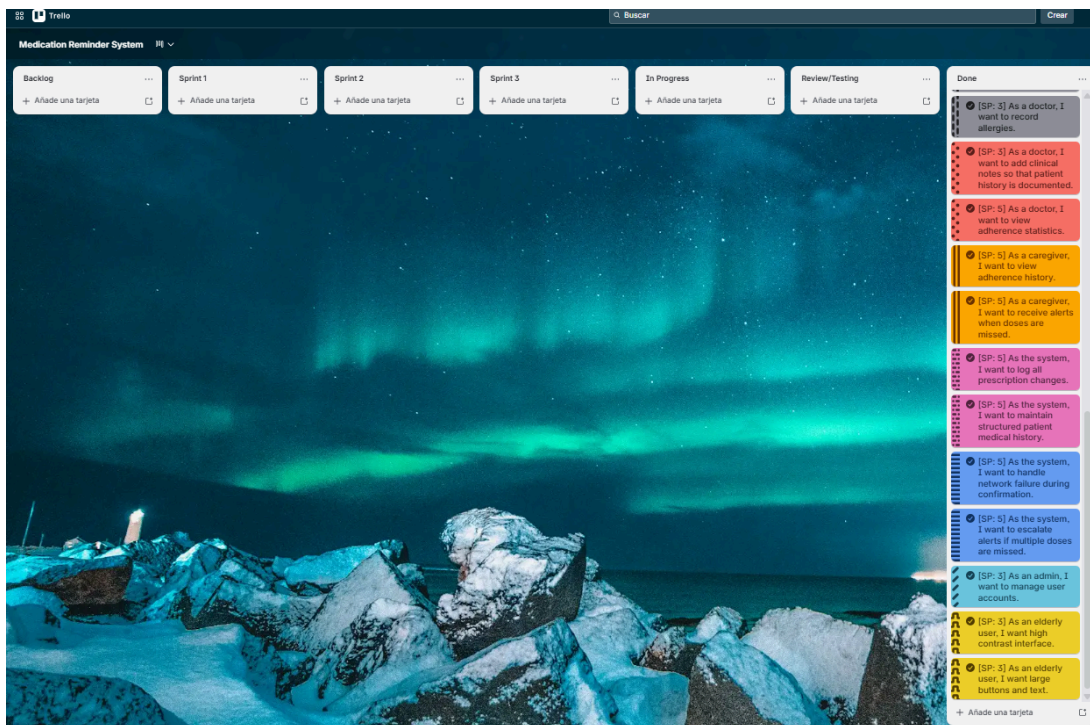


Figure 8: Final State of the Trello Board

This figure represents the final state of the Trello board, where most of the core functionalities have been completed and moved to the "Done" column. It reflects the overall progress of the project, showing a mature stage of development where key system features have been implemented and validated, while remaining tasks are minimal or planned for future iterations.



3. System Architecture

3.1 Hardware Architecture

The system follows a distributed architecture where different hardware components interact through the internet. End users, including elderly patients, caregivers, doctors, and administrators, access the system through personal devices such as smartphones, tablets, or desktop computers. Patients primarily use mobile devices to receive medication reminders and confirm intake, while doctors, caregivers, and administrators interact with the system through web-based dashboards on desktop or laptop devices.

The application is hosted on a server that runs the backend API and manages communication between the frontend interfaces and the database. The database server stores all system data, including user accounts, prescriptions, medication records, and alerts.

In this distributed design, various hardware devices work together over the Internet. End-users such as patients (especially those over 65), caregivers, doctors, and administrators will be able to interact with the system through their personal devices (e.g., smartphones, tablets, or desktop computers). Primarily, patients will use their phones/mobile devices to receive reminders to take their medications and to confirm that they have done so, while all other end-users (doctors, caregivers, administrators) will be using a web dashboard to access the application via desktop/laptop computers.

The system uses a server that hosts the backend API (Application Programming Interface), where it also functions as an intermediary in the communication process between the frontend interfaces and database. The database server is responsible for storing all system data, including end-user accounts, prescriptions and prescription tracking, medication records, and alerts.

3.2. Software Architecture

The architecture of the system is built on a component-based architecture that is composed of layers to maintain separation of concerns, scalability, and maintainability.

At the frontend layer, two main interfaces are provided:

- A **web dashboard** used by doctors, caregivers, and administrators for managing prescriptions, monitoring patient adherence, and managing users.
- A **mobile application** used by elderly patients to receive reminders, view medication schedules, and confirm medication intake.

Both interfaces communicate with the backend through RESTful API calls.

The backend layer is implemented as a server-side application responsible for handling business logic, authentication, validation, and data processing. It includes several key components:

- **Authentication component** for managing user login and access control

- **Prescription management component** for creating and updating prescriptions
- **Medication intake component** for recording medication confirmations
- **Adherence monitoring component** for tracking missed doses
- **Reminder scheduler component** for triggering notifications
- **Alert escalation component** for notifying caregivers when issues occur

The backend interacts with a relational database that stores structured data such as users, patient profiles, prescriptions, medication logs, alerts, and audit records.

In addition, there is an external notification service that is integrated into the overall system that will be responsible for sending out both “reminder” notifications and “alert” notifications for caregivers. This layered approach ensures that the presentation layer, the business logic layer, and the data layer in the system are all separate from each other, leading to better overall reliability and maintainability for the system.

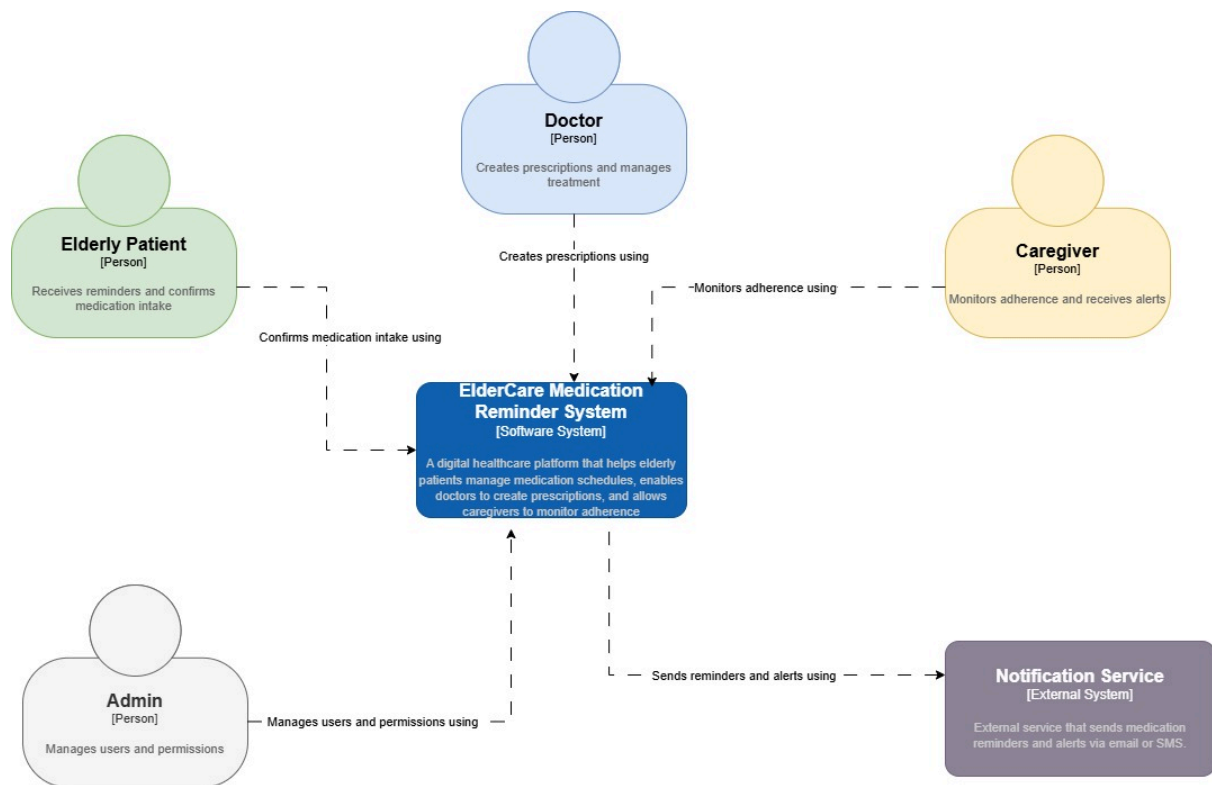
This diagram provides an overview of how this system interacts with all external parties. It shows four main user groups that will use *The ElderCare Medication Reminder System* to perform a specific function:

- **Elderly patients** receive reminders and confirm medication intake
- **Doctors** create prescriptions and manage treatment plans
- **Caregivers** monitor adherence and receive alerts
- **Administrators** manage users and permissions

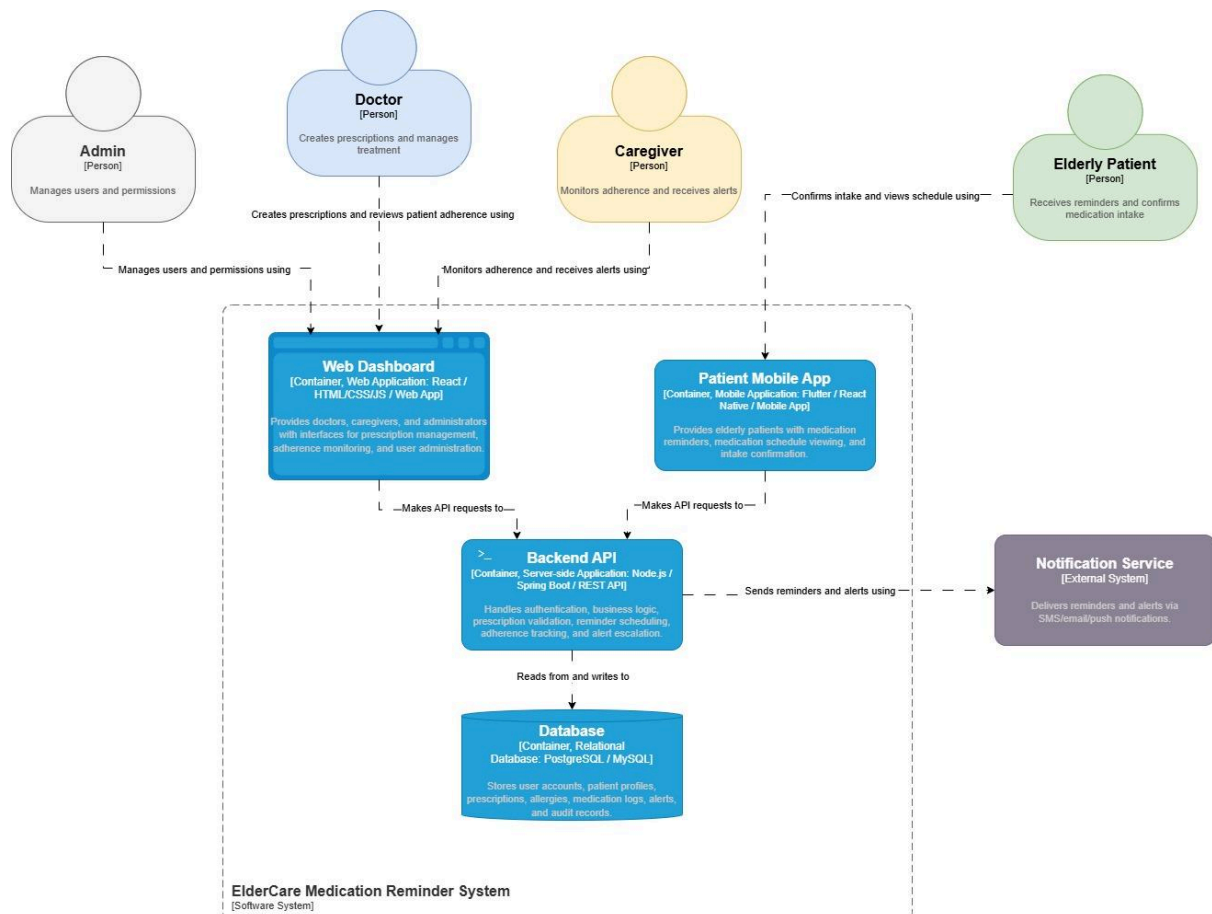
The system also communicates with an external **notification service**.

This diagram highlights the system boundaries and shows how different stakeholders interact with the platform, ensuring that all user roles and external dependencies are clearly defined.

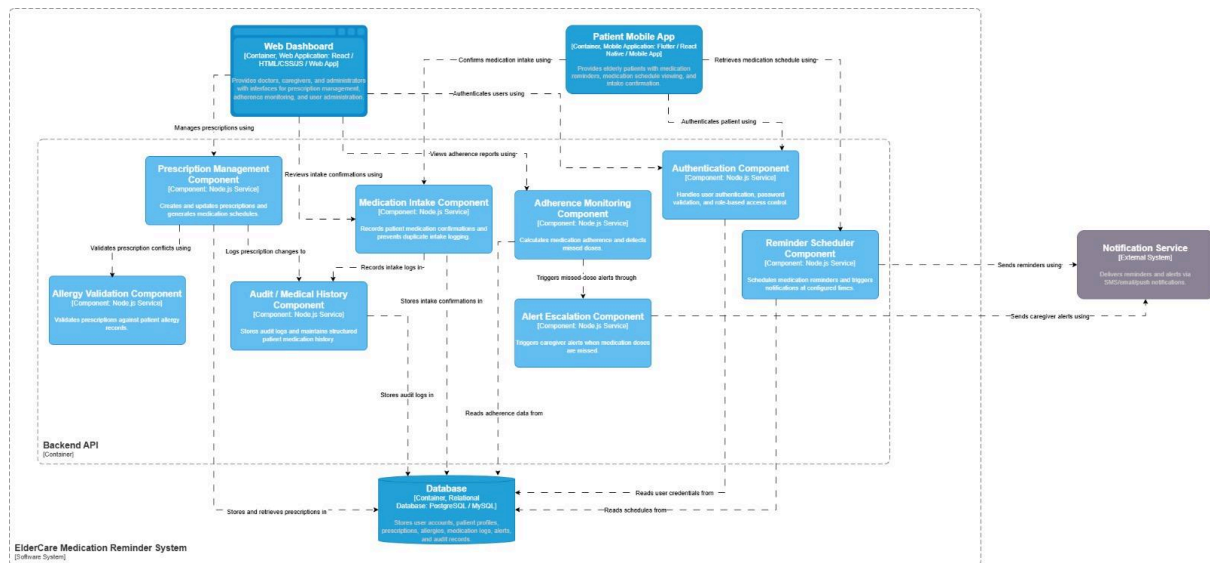
Level 1 System Context Diagram



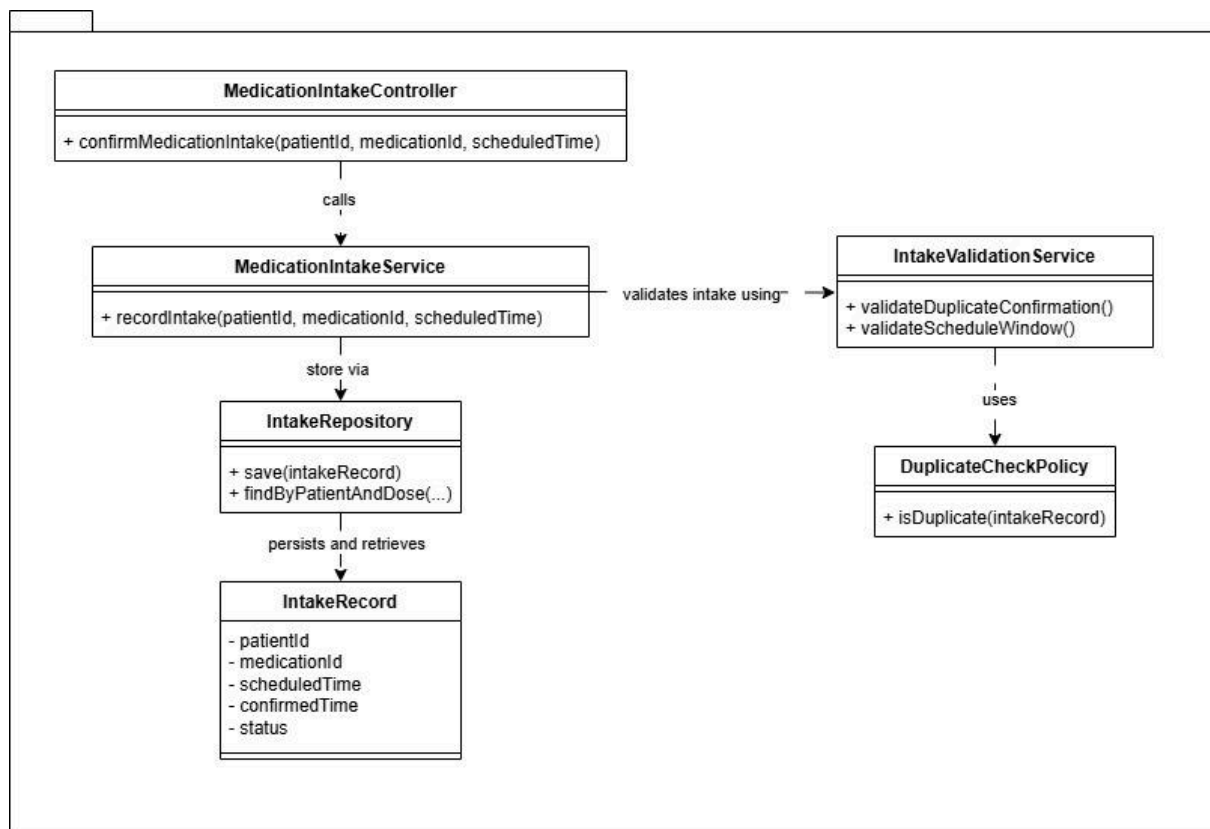
Level 2 Container Diagram



Level 3 Component Diagram



Level 4 Code Diagram



3.2.1.Key interfaces

Method	Endpoint	Description	Request Data	Response
POST	/auth/login	Authenticates user	email, password	JWT token
POST	/auth/register	Creates user account	user data	user created
POST	/prescriptions	Create prescription	medication, dosage, schedule	prescription object
PUT	/prescriptions/{id}	Update prescription	updated data	updated prescription
GET	/patients/{id}/prescriptions	Get patient prescriptions	patientId	prescription list
POST	/intake/confirm	Confirm medication intake	patientId, medicationId, scheduledTime	confirmation status
GET	/patients/{id}/intake-history	Retrieve intake logs	patientId	intake records
GET	/adherence/{patientId}	Get adherence report	patientId	adherence statistics
GET	/alerts/{caregiverId}	Retrieve missed dose alerts	caregiveId	alert list

4. Data Design

4.1 Conceptual Data Model

Core entities and relationships that will help the *ElderCare Medication Reminder* System function are detailed in the conceptual data model. The system is built upon patient-centred data, which will allow for prescription management, medication tracking and alert generation.

The central entity in the system is the User entity, which stores general information such as user ID, name, email, password hash, role, and account creation date. Each user has an assigned role e.g., doctor, caregiver, patient or administrator.

A patient will have a PatientProfile, which will provide additional personal and medical data beyond what is included in the User entity, e.g., date of birth, mailing address, emergency contact information and medical notes. Separate entities for the User and PatientProfile improves data organisation and enables role-based access to data.

A Prescription entity is used to represent prescribed medications by a doctor. It contains attributes of the medications themselves like: medication name, dosage, how often to take it, when to start taking it and when to stop taking it, and any instructions. Each Prescription will be linked to both the Patient's identity, as well as the Doctor's identity to ensure that medical decisions are traceable.

The IntakeRecord entity will be used to track if the medication was taken properly in accordance with its prescription by capturing the actual time of the scheduled intake of medication, the confirmation time of when the intake occurred, and whether or not the intake occurred. The system is able to use this information to ensure that the Patient is taking the prescribed medication, and provide data on adherence.

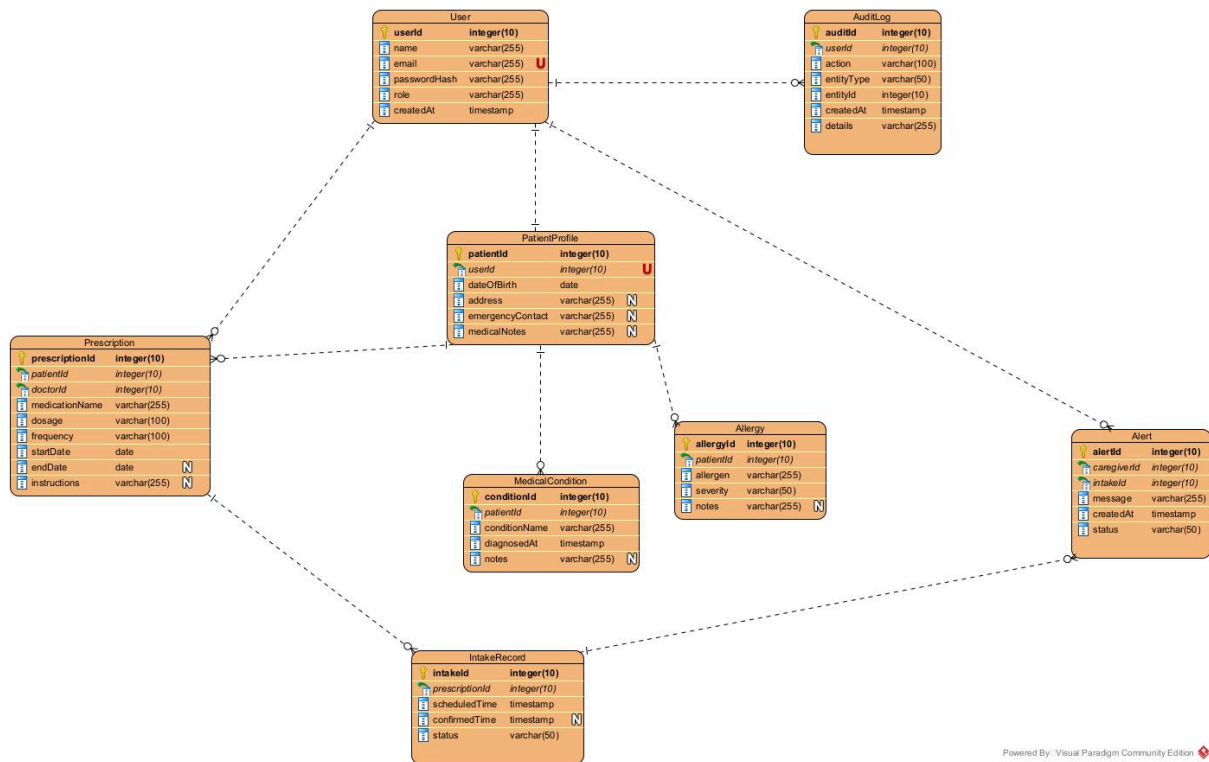
The system has the capability to manage the Patient's health risks through its Allergy Entity. The Allergy Entity stores data related to allergies (e.g. what the allergen is, how severe it is, and any notes). This allows the system to verify the prescriptions to prevent adverse medication interactions.

The system will also store information regarding the Patient's diagnosed Medical Conditions through the MedicalCondition Entity. The MedicalCondition Entity contains attributes such as: name of the Medical Condition, date of diagnosis, and notes. Thus, the MedicalCondition Entity contains valuable context necessary to help make good decisions for treatment and long term care.

An Alert entity provides storage for notifications regarding missed doses and medication reminders. Alerts contain message content, timestamps, status, and links to a patient's caregiver and intake record. Providing real-time alert monitoring and escalation.

In addition, to support accountability and traceability, the AuditLog entity was implemented to capture user actions, entity changes, timestamps, and related information. Capturing this information is very important due to the nature of the security and compliance issues pervasive in health care systems.

In conclusion, the data model provides a clear relationship between users, patients, prescriptions, and medications that will provide an effective means to manage data, increase patient safety, and create dependable system performance.



5. Security Management

Security is important for the *ElderCare Medication Reminder System* because of the confidential nature of patient health information and patient personal information. The system employs many different security features to maintain confidentiality, integrity, and controlled access to medical information.

Authentication and Password Protection

Before a user can log into the platform, they must authenticate with an email and password. Passwords are not stored in plain text; they are hashed and stored in the database, which decreases the possibility of exposing credentials in a data breach.

Role-Based Access Control

The system uses role-based access control to limit functionality to specific users depending on their user type.

All authenticated users are assigned a role, including:

- **Doctor** – can manage prescriptions and patient treatment data
- **Caregiver** – can monitor adherence and receive alerts
- **Patient** – can view schedules and confirm medication intake
- **Administrator** – can manage users and permissions

This ensures users can only access data and actions relevant to their responsibilities.

Audit Logging

The AuditLog entity logs all significant system activity such as creation and editing of prescriptions or confirming of administered medications. Audit logging establishes accountability and traceability for all system activity, and supports future compliance.

Data Validation and Business Rule Protection

Validation of input data is performed throughout all backend services to ensure that only valid and safe data can be processed.

Examples include:

- Preventing prescriptions with missing medication information
- Preventing duplicate medication intake confirmations
- Validating medication intake against the scheduled time window
- Preventing prescriptions that conflict with recorded allergies

Secure Communication

All communication between a front-end applications and a backend API must be done with the HTTPS protocol to encrypt data while in transit and to prevent disclosure of sensitive data.

Future Security Enhancements

Potential future improvements include:

- Multi-factor authentication for doctors and administrators
- JWT token-based session management
- Automatic account lockout after repeated failed login attempts
- Full GDPR / healthcare compliance audit support

6. Testing

The unit tests will be run using **JUnit 5** for the test framework and **Mockito** to mock dependencies such as databases, etc., and perform unit testing on a service without needing to call the actual database.

The services will be developed in isolation from each other; therefore all external dependencies will be mocked and the development will continue to work as it would without the database. Having all services in isolation allows for the tests to be run faster and more reliably.

Both successful scenarios and error cases were covered to validate the robustness of the application.

```
- AuthServiceTest.java
```

The **AuthService** was tested to ensure that the login functionality behaves correctly under different conditions.

The following scenarios were tested:

- Successful login with correct credentials
- Failed login with incorrect password

- Failed login when the user does not exist

```
package com.setu.service;

import com.setu.model.User;
import com.setu.repository.UserRepository;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class AuthServiceTest {

    private UserRepository userRepository;
    private AuthService authService;

    @BeforeEach
    void setUp() {
        userRepository = mock(UserRepository.class);
        authService = new AuthService(userRepository);
    }

    @Test
    void shouldLoginSuccessfully() {
        User user = new User("test@email.com", "1234");

        when(userRepository.findByEmail("test@email.com")).thenReturn(user);

        boolean result = authService.login("test@email.com",
"1234");

        assertTrue(result);
    }

    @Test
    void shouldFailLogin() {
        User user = new User("test@email.com", "1234");

        when(userRepository.findByEmail("test@email.com")).thenReturn(user);
    }
```

```

        boolean result = authService.login("test@email.com",
"wrong");

        assertFalse(result);
    }
    @Test
    void shouldFailLogin_whenUserDoesNotExist() {

        when(userRepository.findByEmail("notfound@email.com"))
            .thenReturn(null);

        boolean result = authService.login("notfound@email.com",
"1234");

        assertFalse(result);
    }
}

```

- PrescriptionServiceTest.java

The `PrescriptionService` was tested to verify the creation and retrieval of prescriptions.

The following scenarios were covered:

- Creating a prescription successfully
- Handling invalid input (missing medication)
- Retrieving prescriptions for a patient
- Returning an empty list when no prescriptions exist
- Verifying that the repository is correctly called

```

package com.setu.service;

import com.setu.model.Prescription;
import com.setu.repository.PrescriptionRepository;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class PrescriptionServiceTest {

    private PrescriptionRepository repository;
    private PrescriptionService service;
}

```

```

@BeforeEach
void setUp() {
    repository = mock(PrescriptionRepository.class);
    service = new PrescriptionService(repository);
}

@Test
void shouldCreatePrescription() {
    Prescription p = new Prescription("Ibuprofen", "2 per
day");

    when(repository.save(p)).thenReturn(p);

    Prescription result = service.createPrescription(p);

    assertNotNull(result);
    assertEquals("Ibuprofen", result.getMedication());
}

@Test
void shouldThrowException_whenMedicationMissing() {
    Prescription p = new Prescription(null, "2 per day");

    assertThrows(IllegalArgumentException.class, () -> {
        service.createPrescription(p);
    });
}

@Test
void shouldReturnMedicationList() {
    List<Prescription> list = List.of(
        new Prescription("Ibuprofen", "2 per day")
    );

    when(repository.findById(1L)).thenReturn(list);

    List<Prescription> result =
service.getPatientMedications(1L);

    assertEquals(1, result.size());
}

@Test
void shouldReturnEmptyList_whenNoMedications() {
    when(repository.findById(1L))
        .thenReturn(List.of());
}

```

```

        List<Prescription> result =
service.getPatientMedications(1L);

        assertTrue(result.isEmpty());
    }
    @Test
    void shouldCallRepository_whenGettingMedications() {

        service.getPatientMedications(1L);

        verify(repository, times(1)).findByPatientId(1L);
    }
}

```

- MedicationIntakeServiceTest.java

The **MedicationIntakeService** was tested to verify the medication intake confirmation workflow and ensure that intake validations are correctly applied before storing intake records.

The following scenarios were covered:

- Confirming medication intake successfully
- Preventing duplicate medication confirmations
- Rejecting confirmations outside the valid schedule window
- Verifying that intake records are persisted only after successful validation

```

package com.setu.service;

import com.setu.model.IntakeRecord;
import com.setu.repository.IntakeRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class MedicationIntakeServiceTest {

    private IntakeRepository intakeRepository;
    private IntakeValidationService validationService;
    private MedicationIntakeService medicationIntakeService;

    @BeforeEach
    void setUp() {
        intakeRepository = mock(IntakeRepository.class);
        validationService = mock(IntakeValidationService.class);

        medicationIntakeService =

```

```

        new MedicationIntakeService(intakeRepository,
validationService);
    }

    @Test
    void shouldConfirmMedicationIntakeSuccessfully() {
        IntakeRecord intake =
            new IntakeRecord(1L, 101L, "2025-05-20T08:00");

        when(intakeRepository.save(intake)).thenReturn(intake);

        IntakeRecord result =

medicationIntakeService.confirmMedicationIntake(intake);

        assertNotNull(result);

        verify(validationService).validateDuplicateConfirmation(intake);
        verify(validationService).validateScheduleWindow(intake);
        verify(intakeRepository).save(intake);
    }

    @Test
    void shouldThrowExceptionWhenDuplicateIntakeDetected() {
        IntakeRecord intake =
            new IntakeRecord(1L, 101L, "2025-05-20T08:00");

        doThrow(new IllegalArgumentException("Duplicate intake"))
            .when(validationService)
            .validateDuplicateConfirmation(intake);

        assertThrows(
            IllegalArgumentException.class,
            () ->
medicationIntakeService.confirmMedicationIntake(intake)
        );

        verify(intakeRepository, never()).save(any());
    }

    @Test
    void shouldThrowExceptionWhenOutsideScheduleWindow() {
        IntakeRecord intake =
            new IntakeRecord(1L, 101L, "2025-05-20T08:00");

        doThrow(new IllegalArgumentException("Outside schedule"))
            .when(validationService)

```

```

        .validateScheduleWindow(intake);

        assertThrows(
            IllegalArgumentException.class,
            () ->
medicationIntakeService.confirmMedicationIntake(intake)
        );

        verify(intakeRepository, never()).save(any());
    }
}

```

7. Prototyping

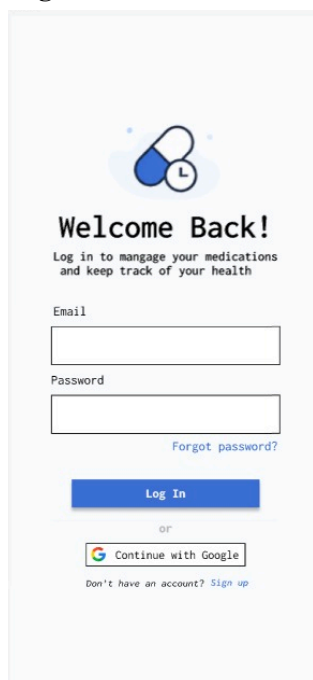
7.1 Prototype Overview

An interactive prototype with a high level of fidelity was created in Figma to visualize the user interface and user workflows prior to creating the *ElderCare Medication Reminder System*.

The prototype illustrated expected interactions with patients, caregivers, and doctors and was used to confirm the usability and workflow design of the system.

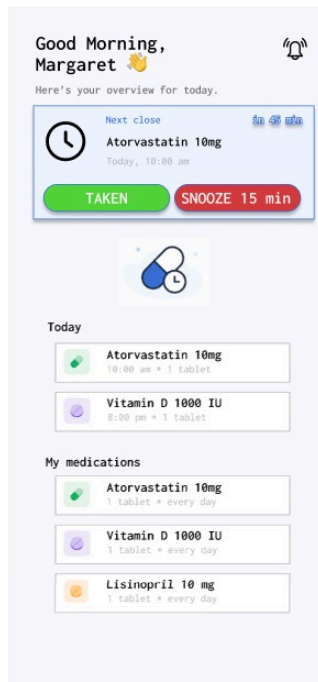
7.2 Patient Mobile Application Screens

Login Screen



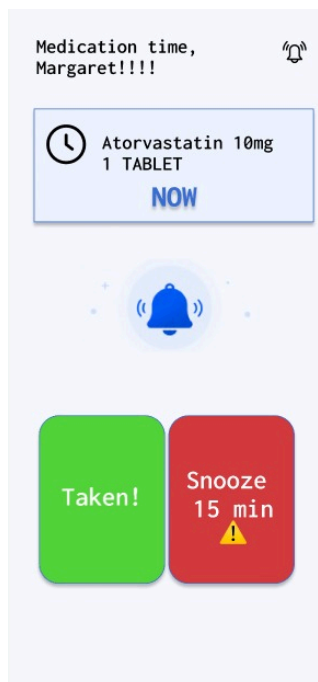
Allows patients to authenticate using email/password or Google Sign-In before accessing medication schedules and reminders.

Patient Dashboard



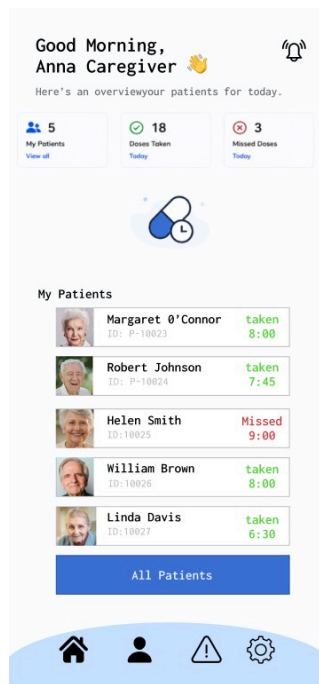
Displays upcoming medication reminders, today's schedule, and active prescriptions. Patients can confirm medication intake or snooze reminders directly from the dashboard.

Medication Reminder Alert



Displays real-time reminder notification when medication is due, allowing the patient to mark the dose as taken or snooze for 15 minutes.

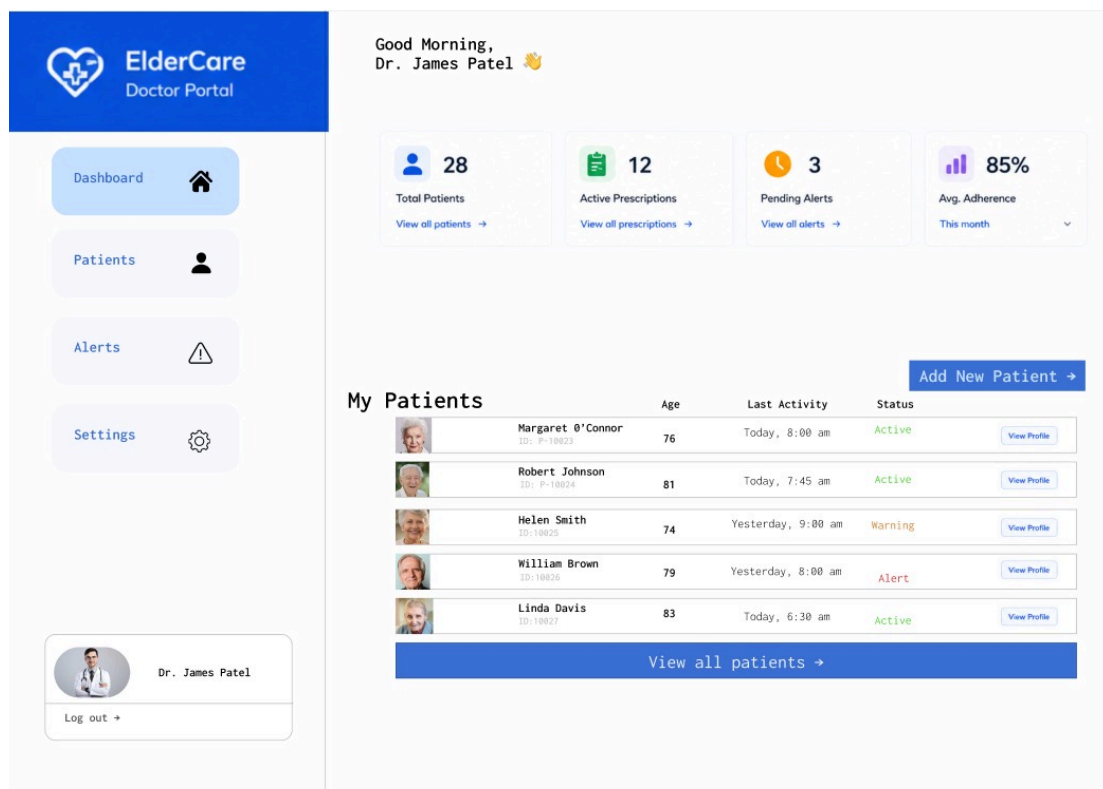
7.3 Caregiver Dashboard



Provides caregivers with an overview of assigned patients, medication adherence statistics, and missed dose alerts.

7.4 Doctor Web Portal

Doctor Dashboard



Displays patient overview, adherence analytics, and pending alerts for all monitored patients.

Patient Medical Profile

 ElderCare
Doctor Portal

Dashboard

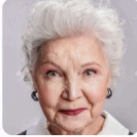
Patients

Alerts

Settings

 Dr. James Patel
Log out →

Patients > Margaret O'Connor



Born: May 12, 1948
ID: P-10023 * 76 YEARS * FEMALE

Gender
Female

Age
76 years

Phone
+353 87 123 4567

Address
12 Greenview Rd,
Dublin, Ireland

Overview

Prescriptions

Adherence

Notes

History

Allergies

• Penicillin
• Ibuprofen

Medical Conditions

• Hypertension
• Type 2 Diabetes
• Osteoarthritis

Current Medications

Atorvastatin 10mg
1 tablet * every day

Vitamin D 1000 IU
1 tablet * every day

Lisinopril 10 mg
1 tablet * every day

Add New Prescription

Adherence Summary (This Month)

85%
Overall

17
Taken

3
Missed

2
Skipped

View Adherence Details

Allows doctors to review allergies, medical conditions, medication history, and adherence metrics for an individual patient.

Prescription Creation Interface

 ElderCare
Doctor Portal

Dashboard

Patients

Alerts

Settings

 Dr. James Patel
Log out →

Patients > Margaret O'Connor



Born: May 12, 1948
ID: P-10023 * 76 YEARS * FEMALE

Gender
Female

Age
76 years

Phone
+353 87 123 4567

Address
12 Greenview Rd,
Dublin, Ireland

CREATE PRESCRIPTION

Prescription details

Medication Name

Enter medication name...

Dosage

Eg 10 mg, 500mg, 1 tablet

Frequency

select frequency

Time(s)

e.g 8:00 am

add +

Start Date

May 24, 2025

End Date (Optional)

May 24, 2025

Duration (Optional)

e.g. 7 days

Notes

Add any additional notes or instructions for the patient...

Prescription Guidelines

• Ensure medication name and dosage are accurate.

• Specify frequency and times clearly.

• Add notes for any special instructions for the patient.

Current Medications

Atorvastatin 10mg
1 tablet * every day

Vitamin D 1000 IU
1 tablet * every day

Lisinopril 10 mg
1 tablet * every day

32

Enables doctors to create new prescriptions by specifying medication details, dosage, schedule, duration, and additional instructions.

7.5 Prototype Design Rationale

The interface was designed with accessibility and elderly usability in mind:

- Large buttons for medication confirmation
- High-contrast colour coding for reminders and alerts
- Minimal navigation complexity
- Clear separation of doctor, caregiver, and patient workflows

8. Conclusion

This project has resulted in the successful design and architecture of a digital health platform intended to help improve medication adherence by elderly patients, and to provide high quality support to doctors and caregivers regarding the monitoring of treatment outcomes.

Throughout this project we completed a full software design lifecycle which included: requirements analysis, prototyping, system architecture design, database modelling, component decomposition, and unit testing of core application logic. We created high fidelity user interface prototypes for all target users (patients, caregivers, and medical professionals) to validate the ease of use and effectiveness of the platform.

We built the system architecture using the C4 model so as to present an easily understood view of the software across context, container, component, and code views. We also developed an ER Diagram to provide a model of the relationships among users, patients, prescriptions, intake records, alerts, allergies, and audit logs. We incorporated security-related items into the design to provide assurance that sensitive health data will be protected. These security-related items include: authentication, role-based access control, audit logging, and data validation.

We implemented unit tests on important service areas, such as authentication, prescription management, and medication intake confirmation, in order to ensure that the core application logic is functioning correctly and reliably.

The medication reminder platform has been designed with an architecture that can be easily scaled and maintained and is capable of supporting real-world healthcare processes. Enhancements to the platform in the future could include additional integration with wearable devices, machine-learning based adherence prediction, advanced analytics dashboards, and additional features for compliance with healthcare regulations.